МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМ. ТАРАСА ШЕВЧЕНКА
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра мережевих та інтернет технологій

СИСТЕМНЕ ПРОГРАМУВАННЯ

Лабораторна робота №3

ПАРАЛЕЛІЗМ ДАНИХ ТА ПАРАЛЕЛІЗМ ЗАДАЧ

ЧАСТИНА ІІ

Довгодька Дениса Станіславовича

Мета: поглибити знання з паралелізму даних та задач на прикладі платформи C# .NET

Хід роботи

Завдання 1.

Створити програму, що використовує для розпаралелювання метод Parallel.For(). Провести ряд обчислювальних експериментів із різним типом елементів масиву (int, double); різною кількістю елементів масиву; різною складністю обчислень:

$$\mathrm{x} = x/10,$$
$$\mathrm{x} = x\ /\ \pi,$$
$$\mathrm{x} = e^{x}\ /\ \mathrm{x}^{\pi},$$
$$\mathrm{x} = e^{\pi x}\ /\ \mathrm{x}^{\pi},$$

Виміряти час, що витрачається паралельним способом обробки та послідовним. Результати експериментів оформити у вигляді таблиці. Зробити висновки.

Створимо програмний застосунок, який реалізує задачу. Вивід в консоль відображено на рисунку 3.1.

```
Select command to investigate Parallel:
1 - Parallel.For() experiments with different array types and sizes
2 - Parallel.For() with exit condition
3 - Parallel.ForEach()
4 - Parallel.ForEach() with lambda expression
0 - Exit
Your choice: 1
Type    Size    Formula Serial(s)       Parallel(s)     SpeedUp
double  100000  x/10    0,0019          0,0543          0,04
int     100000  x/10    0,0017          0,0012          1,39
double  100000  x/pi    0,0030          0,0089          0,33
int     100000  x/pi    0,0039          0,0007          5,38
double  100000  e^x/x^pi        0,0078          0,0029          2,67
int     100000  e^x/x^pi        0,0063          0,0044          1,43
double  100000  e^(pi*x)/x^pi   0,0273          0,0036          7,62
int     100000  e^(pi*x)/x^pi   0,0186          0,0029          6,34
double  1000000 x/10    0,0439          0,0034          12,82
int     1000000 x/10    0,0145          0,0034          4,24
double  1000000 x/pi    0,0174          0,0042          4,11
int     1000000 x/pi    0,0196          0,0042          4,71
double  1000000 e^x/x^pi        0,1502          0,0281          5,34
int     1000000 e^x/x^pi        0,1496          0,0249          6,01
double  1000000 e^(pi*x)/x^pi   0,2838          0,0350          8,11
int     1000000 e^(pi*x)/x^pi   0,1879          0,0270          6,95
Press enter to continue
```

Рисунок 3.1 – Виведення таблиці витраченого часу алгоритмом

З отриманих результатів видно, що паралельне виконання дає суттєве прискорення для складних обчислень і великих масивів, тоді як для простих операцій на невеликих масивах ефект може бути незначним або навіть гіршим за послідовне виконання.

Завдання 2.

Модифікувати попередню програму (створивши новий проект у рішенні) таким чином, щоб відбувався вихід з паралельного циклу за умови входження значення елемента у деякий окіл деякого числа (число та відхилення задаються константами).

Маємо наступний вивід із алгоритму (рис. 3.2)

```
Select command to investigate Parallel:
1 - Parallel.For() experiments with different array types and sizes
2 - Parallel.For() with exit condition
3 - Parallel.ForEach()
4 - Parallel.ForEach() with lambda expression
0 - Exit
Your choice: 2
Parallel.For() with exit condition demo

using break()
[Break] Found value 52 at index 52
Finished indices: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27,
28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 56, 57, 58, 59, 64,
65, 66, 67, 68, 72, 73, 74, 75, 76, 80, 81, 82, 83, 84, 88, 89, 90, 96, 97, 98
Loop stopped at iteration 52
Time: 143 ms

next

Using stop()
[Stop] Found value 52 at index 52
Finished indices: 0, 1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 16, 17, 18, 19, 20, 24, 25, 26, 27, 28, 32, 33, 34, 35, 36, 40, 41
, 42, 43, 44, 48, 49, 50, 51, 52, 56, 57, 58, 59, 60, 64, 65, 66, 67, 68, 72, 73, 74, 75, 76, 80, 81, 82, 83, 84, 88, 89
, 90, 91, 92, 96, 97, 98, 99
Loop was stopped early
Time: 71 ms
Press enter to continue
```

Рисунок 3.2 – Вивід чисел, для яких задачі встигли початись до знайденого елемента – набір опрацьованих індексів

Завдання 3.

Повторити приклади розпаралелювання за допомогою ForEach().

Отримаємо наступний вивід результату виконання створеного алгоритму для вирішення задачі (рис. 3.3)

```
Select command to investigate Parallel:
1 - Parallel.For() experiments with different array types and sizes
2 - Parallel.For() with exit condition
3 - Parallel.ForEach()
4 - Parallel.ForEach() with lambda expression
0 - Exit
Your choice: 3
Parallel.ForEach() with exit condition demo

[Break] Finished indices:
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32
, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 56, 57, 58, 64, 65, 66, 72, 73, 74, 80, 81, 82
, 88, 96

next

[Stop] Finished indices:
0, 1, 2, 3, 4, 6, 8, 9, 10, 16, 17, 24, 25, 26, 32, 33, 34, 40, 41, 42, 48, 49, 50, 56, 57, 58, 59, 64, 65, 66, 67, 72,
73, 74, 75, 80, 81, 82, 83, 88, 89, 90, 96, 97, 98, 99
Press enter to continue
```

Рисунок 3.3 – Результат роботи алгоритму

Завдання 4.

Модифікувати попередню програму (створивши новий проєкт у рішенні) таким чином, щоб тіло паралельного циклу задавалося лямбда-виразом.

Виведемо результат роботи програми (рис. 3.4)

```
Select command to investigate Parallel:
1 - Parallel.For() experiments with different array types and sizes
2 - Parallel.For() with exit condition
3 - Parallel.ForEach()
4 - Parallel.ForEach() with lambda expression
0 - Exit
Your choice: 4
Parallel.ForEach() with exit condition demo

[Break] Finished indices:
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32
, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 56, 57, 58, 64, 65, 72, 73, 74, 80, 81, 82, 88
, 96

next

[Stop] Finished indices:
0, 1, 8, 9, 10, 16, 17, 24, 25, 26, 32, 33, 34, 40, 41, 48, 49, 50, 56, 57, 58, 64, 65, 72, 73, 74, 80, 81, 82, 88, 89,
96, 97, 98
Press enter to continue
```

Рисунок 3.4 – Виконання алгоритмічних задач із Parallel.ForEach та лямбда виразом

Висновок: В ході лабораторної роботи було досліджено роботу циклів For і ForEach у поєднанні з паралельним виконанням та використанням Break і Stop. Показано, що Break зупиняє лише внутрішній цикл, тоді як Stop припиняє виконання всієї задачі або потоку. Parallel.Invoke і Task дозволяють організувати паралельне виконання коду, ефективно керуючи потоками і підвищуючи продуктивність програми.